

FRED TALKS

17th April 2026

CONTENTS

Seeing like an agent: how we design tools in Claude Code

CLAUDE · 1540 WORDS

3-2-1: On acting like a winner, thinking for yourself, and how to stop chasing what isn't working

JAMES CLEAR · 590 WORDS

Introducing Claude Opus 4.7

ANTHROPIC NEWS · 1068 WORDS

Seeing like an agent: how we design tools in Claude Code

CLAUDE · 12 APR 2026 · [SOURCE](#)

One of the hardest parts about building an agent harness is constructing its tools.

Claude acts completely through [tool calling](#), but there are a number of ways tools can be constructed in the Claude API with primitives like [bash](#), [skills](#) and [code execution](#). (You can read more about programmatic tool calling on the Claude API in [@RLanceMartin's new article](#)).

So how do you design your agents' tools? Do you give it one general-purpose tool like bash or code execution? Or fifty specialized tools, one for each use case?

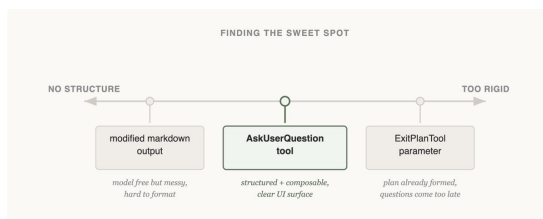
To put yourself in the mind of the model, imagine being given a difficult math problem. What tools would you want in order to solve it? It would depend on your own skill set!

Paper would be the minimum, but you'd be limited by manual calculations. A calculator would be better, but you would need to know how to operate the more advanced options. The fastest and most powerful option would be a computer, but you would have to know how to use it to write and execute code.

This is a useful framework for designing your agent. You want to give it tools that are shaped to its own abilities. But how do you know what those abilities are? You pay attention, read its outputs, experiment. You learn to see like an agent.

If you're building an agent, you'll face the same questions we did: when to add a tool, when to remove one, and how to tell the difference. Here's how we've answered them while building Claude Code, including where we got it wrong first.

Improving elicitation with the AskUserQuestion tool



When building the AskUserQuestion tool, our goal was to improve Claude’s ability to ask questions (often called elicitation).

While Claude could just ask questions in plain text, we found answering those questions felt like they took an unnecessary amount of time. How could we lower this friction and increase the bandwidth of communication between the user and Claude?

Attempt 1: Editing the ExitPlanTool

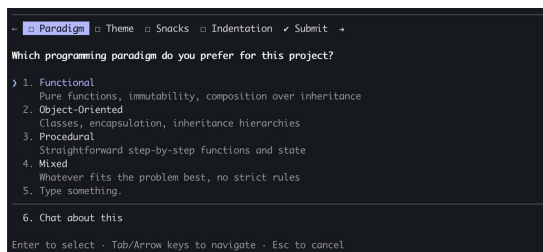
The first approach we tried was adding a parameter to the ExitPlanTool to have an array of questions alongside the plan. This was the easiest fix to implement, but it confused Claude because we were simultaneously asking for a plan and a set of questions about the plan. What if the user’s answers conflicted with what the plan said? Would Claude need to call the ExitPlanTool twice? We knew this tactic wouldn’t work, so we went back to the drawing board. (You can read more about why we made an ExitPlanTool in [our post on prompt caching](#))

Attempt 2: Changing output format

Next, we tried updating Claude’s output instructions to serve a slightly modified markdown format that it could use to ask questions. For example, we could ask it to output a list of bullet point questions with alternatives in brackets. We could then parse and format that question as UI for the user.

Claude could usually produce this format, but not reliably. It would append extra sentences, drop options, or abandon the structure altogether. Onto the next approach.

Attempt 3: The AskUserQuestion Tool



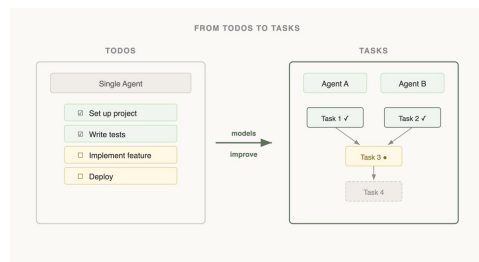
Finally, we landed on creating a tool that Claude could call at any point, but it was particularly prompted to do so during plan mode. When the tool triggered we would show a modal to display the questions and block the agent's loop until the user answered.

This tool allowed us to prompt Claude for a structured output and it helped us ensure that Claude gave the user multiple options. It also gave users ways to compose this functionality, for example calling it in the [Agent SDK](#) or using referring to it in skills.

Most importantly, Claude seemed to like calling this tool and we found its outputs worked well. After all, even the best designed tool doesn't work if Claude doesn't understand how to call it.

Is this the final form of elicitation in Claude Code? We doubt it. As Claude gets more capable, the tools that serve it have to evolve too. The next section shows a case where a tool that once helped started getting in the way.

Updating with capabilities: tasks & todos



When we first launched Claude Code, we realized that the model needed a todo list to keep it on track. Todos could be written at the start and checked off as the model did work. To do this we gave Claude the `TodoWrite` tool, which would write or update Todos and display them to the user.

But even then, we often saw Claude forgetting what it had to do. To adapt, we inserted system reminders every 5 turns that reminded Claude of its goal.

As models improved, they found To-do lists limiting. Being sent reminders of the todo list made Claude think that it had to stick to the list instead of modifying it when it realized it needed to change course. We also saw Opus 4.5 also get much better at using subagents, but how could subagents coordinate on a shared todo list?

Seeing this, we replaced the `TodoWrite` feature with [the Task tool](#). Whereas todos are focused on keeping the model on track, tasks help agents communicate with each other. Tasks could include dependencies, share updates across subagents and the model could alter and delete them.

As model capabilities increase, the tools that your models once needed might now be constraining them. It's important to constantly revisit previous assumptions on what tools are needed. This is also why it's useful to stick to a small set of models to support that have a fairly similar capabilities profile.

Designing a search interface

The most consequential tools we've built are the ones that let Claude find its own context.

When Claude Code was first released internally, we used RAG: a vector database would pre-index the codebase, and the harness would retrieve relevant snippets and hand them to Claude before each response.. While RAG was powerful and fast, it required indexing and setup and could be fragile across a host of different environments. Most importantly, Claude was *given* this context instead of finding the context itself.

But if Claude could search on the web, why couldn't it also search your codebase? By giving Claude a Grep tool, we could let it search for files and build context itself.

As Claude gets smarter, it becomes increasingly good at building its context when given the right tools.

When we introduced Agent Skills, we formalized the idea of progressive disclosure, which allows agents to incrementally discover relevant context through exploration.

Claude could now read skill files and those files could then reference other files that the model could read recursively. In fact, a common use of skills is to add more search capabilities to Claude like giving it instructions on how to use an API or query a database.

Over the course of a year, Claude went from not really being able to build its own context to being able to do nested search across several layers of files to find the exact context it needed.

Progressive disclosure is now a common technique we use to add new functionality without adding a tool. In the next section, we explain why.

Progressive disclosure: the Claude Code Guide agent

Claude Code currently has ~20 tools, and our team frequently revisits if we need all of them for Claude to be most effective. The bar to add a new tool is high, because this gives the model one more option to think about.

For example, we noticed that Claude did not know enough about how to use Claude Code. If you asked it how to add a MCP or what a slash command did, it would not be able to reply.

We could have put all of this information in the system prompt, but given that users rarely asked about this, it would have added context rot and interfered with Claude Code's main job: writing code.

Instead, we tried progressive disclosure: we gave Claude a link to its docs that it could load and search when needed. This worked, but Claude would pull large chunks of documentation into context to find an answer the user could have gotten in one sentence.

So we built the Claude Code Guide — a subagent Claude calls whenever a user asks about Claude Code itself. The subagent does the doc-searching in its own context, follows detailed instructions on how to search and what to extract, and hands back only the answer. The main agent's context stays clean.

While this isn't a perfect solution (Claude can still get confused when you ask it about how to set itself up), we were able to add things to Claude's action space without adding a new tool.

Seeing like an agent is an art, not a science

Designing the tools for your models is as much an art as it is a science. It depends heavily on the model you're using, the goal of the agent and the environment it's operating in.

Our best advice? Experiment often, read your outputs, try new things. And most importantly, try to see like an agent.

***About the author:** Thariq Shihpar is a member of technical staff at Anthropic, working on Claude Code.*

3-2-1: On acting like a winner, thinking for yourself, and how to stop chasing what isn't working

JAMES CLEAR · 16 APR 2026 · [SOURCE](#)

3-2-1 Thursday *by* JAMES CLEAR

*“The most wisdom per word of any
newsletter on the web.”*

3-2-1: On acting like a winner, thinking for yourself, and how to stop chasing what isn't working

read on

JAMESCLEAR.COM | APRIL 16, 2026

Happy 3-2-1 Thursday!

Here are 3 ideas, 2 quotes, and 1 question to consider this week...

3 Ideas From Me

I.

"Big outcome: Do the right thing well.

Big lesson: Do the right thing poorly.

Big risk: Do the wrong thing well."

II.

"So much of what we believe is the residue of someone else's thinking.

Pause and question things for a moment. Is this really how it has to be? Is this really what you want?"

III.

"It's not a race.

You are not ahead. You are not behind. You are here.

Enjoy it and make the most of it."

2 Quotes From Others

I.

Financial investor **Warren Buffett** encourages us not to cling to our losing efforts or chase what isn't working and instead focus on the next best action:

"You don't have to make it back the way that you lost it."

Source: 1994 Berkshire Hathaway shareholder letter

II.

Football coach **Bill Walsh**, winner of three Super Bowls, reminds us that standards come first and results follow:

"Champions behave like champions before they're champions; they have a winning standard of performance before they are winners."

Source: The Score Takes Care of Itself

1 Question For You

If you lived in a world where everyone made the exact same salary, what would you choose to do for work?

Want to share this issue of 3-2-1? Just copy and paste this link:

<https://jamesclear.com/3-2-1/april-16-2026>

Until next week,

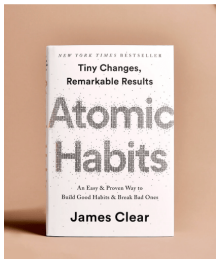
James Clear

Author of Atomic Habits

Cofounder of Authors Equity

p.s. How to start an email .

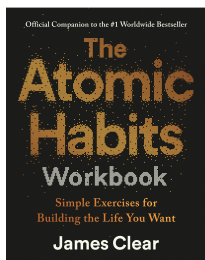
What else am I working on?



Get my book

Read my #1 New York Times bestseller, Atomic Habits , which has sold more than 25 million copies worldwide.

[Click here to learn more](#)



The Atomic Habits Workbook

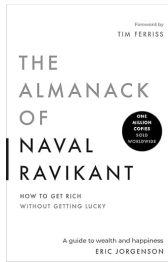
The official companion to Atomic Habits. Filled with simple exercises for building the life you want.

[Click here to learn more](#)

My book recommendations this week

In addition to my own writing, I cofounded a publishing company called Authors Equity to help other authors publish their books. As a result, I get a front row seat to a lot of great books.

Here are a few books I think you'll love:



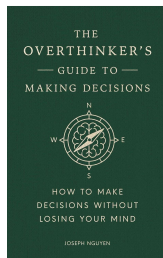
OVER 1 MILLION COPIES SOLD

Almanack of Naval Ravikant

How to get rich without getting lucky

by Eric Jorgenson

Print . Audio . Ebook



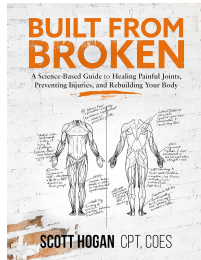
NEW RELEASE

The Overthinker's Guide to Making Decisions

How to make decisions without losing your mind

by Joseph Nguyen

Print . Audio . Ebook



ORDER NOW

Built from Broken

A science-based guide to healing painful joints, preventing injuries, and rebuilding your body

by Scott Hogan

Print . Audio . Ebook

Follow me on social:



About this newsletter: You are receiving this email because you subscribed to my weekly 3-2-1 newsletter. Every Thursday, I share 3 ideas from me, 2 quotes from others, and 1 question for you to ponder. Occasionally, I send out longer content on habits and self-improvement.

Update your subscription preferences: Unsubscribe from the 3-2-1 newsletter , unsubscribe from all emails (including future book announcements) , or manage your subscriber profile .

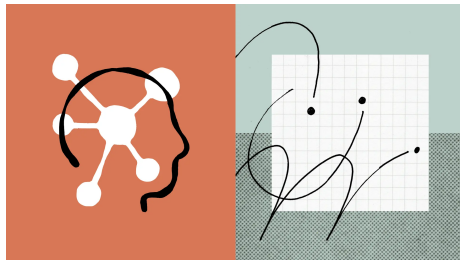
Atomic Habits customers: [Click here to learn how to redeem your purchase for free bonuses.](#) This will also remove you from any promotional emails for the book.

2935 E Main St. Unit #9361, Columbus, OH 43209



Introducing Claude Opus 4.7

ANTHROPIC NEWS · 16 APR 2026 · [SOURCE](#)



Our latest model, Claude Opus 4.7, is now generally available.

Opus 4.7 is a notable improvement on Opus 4.6 in advanced software engineering, with particular gains on the most difficult tasks. Users report being able to hand off their hardest coding work—the kind that previously needed close supervision—to Opus 4.7 with confidence. Opus 4.7 handles complex, long-running tasks with rigor and consistency, pays precise attention to instructions, and devises ways to verify its own outputs before reporting back.

The model also has substantially better vision: it can see images in greater resolution. It's more tasteful and creative when completing professional tasks, producing higher-quality interfaces, slides, and docs. And—although it is less broadly capable than our most powerful model, Claude Mythos Preview—it shows better results than Opus 4.6 across a range of benchmarks:

	Open 17	Open 16	Q27-14	Control 14/16	Significance
Agentic working memory	47.4%	47.4%	52.2%	52.2%	NS
Agentic working memory (L2)	47.4%	47.4%	—	47.4%	NS
Agentic operational memory	47.4%	47.4%	75.0%	75.0%	0.028
Agentic operational memory (L2)	47.4%	47.4%	—	47.4%	NS
Intelligence (g)	41.0%	41.0%	41.7%	41.6%	NS
Intelligence (g) (L2)	41.0%	41.0%	—	41.0%	NS
Agentic search	51.0%	51.0%	69.0%	69.0%	0.017
Agentic search (L2)	51.0%	51.0%	—	51.0%	NS
Agentic comprehension	53.0%	53.0%	68.0%	68.0%	—
Agentic comprehension (L2)	53.0%	53.0%	—	53.0%	NS
Agentic problem solving	64.0%	64.0%	80.0%	80.0%	—
Agentic problem solving (L2)	64.0%	64.0%	—	64.0%	NS
Intelligence (g)	73.0%	73.0%	69.0%	—	0.016
Intelligence (g) (L2)	73.0%	73.0%	—	73.0%	NS
Intelligence (g)	54.0%	54.0%	54.0%	54.0%	NS
Intelligence (g) (L2)	54.0%	54.0%	—	54.0%	NS
Working memory	40.0%	40.0%	—	—	NS
Working memory (L2)	40.0%	40.0%	—	—	NS
Intelligence (g)	40.0%	40.0%	—	40.0%	NS

Last week we announced [Project Glasswing](#), highlighting the risks—and benefits—of AI models for cybersecurity. We stated that we would keep Claude Mythos Preview’s release limited and test new cyber safeguards on less capable models first. Opus 4.7 is the first such model: its cyber capabilities are not as advanced as those of Mythos Preview (indeed, during its training we experimented with efforts to differentially reduce these capabilities). We are releasing Opus 4.7 with safeguards that automatically detect and block requests that indicate prohibited or high-risk cybersecurity uses. What we learn from the real-world deployment of these safeguards will help us work towards our eventual goal of a broad release of Mythos-class models.

Security professionals who wish to use Opus 4.7 for legitimate cybersecurity purposes (such as vulnerability research, penetration testing, and red-teaming) are invited to join our new [Cyber Verification Program](#).

Opus 4.7 is available today across all Claude products and our API, Amazon Bedrock, Google Cloud's Vertex AI, and Microsoft Foundry. Pricing remains the same as Opus 4.6: \$5 per million input tokens and \$25 per million output tokens. Developers can use `claude-opus-4-7` via the Claude API.

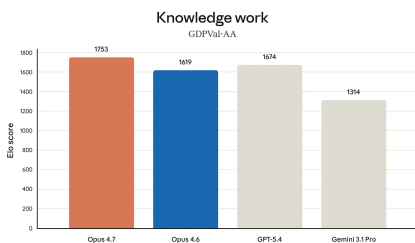
Testing Claude Opus 4.7

Claude Opus 4.7 has garnered strong feedback from our early-access testers:

Below are some highlights and notes from our early testing of Opus 4.7:

- *Instruction following.* Opus 4.7 is substantially better at following instructions. Interestingly, this means that prompts written for earlier models can sometimes now produce unexpected results: where previous models interpreted instructions loosely or skipped parts entirely, Opus 4.7 takes the instructions literally. Users should re-tune their prompts and harnesses accordingly.
- *Improved multimodal support.* Opus 4.7 has better vision for high-resolution images: it can accept images up to 2,576 pixels on the long edge (~3.75 megapixels), more than three times as many as prior Claude models. This opens up a wealth of multimodal uses that depend on fine visual detail: computer-use agents reading dense screenshots, data extractions from complex diagrams, and work that needs pixel-perfect references.¹
- *Real-world work.* As well as its state-of-the-art score on the Finance Agent evaluation (see table above), our internal testing showed Opus 4.7 to be a more effective finance analyst than Opus 4.6, producing rigorous analyses and models, more professional presentations, and tighter integration across tasks. Opus 4.7 is also state-of-the-art on GDPval-AA, a third-party evaluation of economically valuable knowledge work across finance, legal, and other domains.
- *Memory.* Opus 4.7 is better at using file system-based memory. It remembers important notes across long, multi-session work, and uses them to move on to new tasks that, as a result, need less up-front context.

The charts below display more evaluation results from our pre-release testing, across a range of different domains:



Safety and alignment

Overall, Opus 4.7 shows a similar safety profile to Opus 4.6: our evaluations show low rates of concerning behavior such as deception, sycophancy, and cooperation with misuse. On some measures, such as honesty and resistance to malicious “prompt injection” attacks, Opus 4.7 is an improvement on Opus 4.6; in others (such as its tendency to give overly detailed harm-reduction advice on controlled substances), Opus 4.7 is modestly weaker. Our alignment assessment concluded that the model is “largely well-aligned and trustworthy, though not fully ideal in its behavior”. Note that Mythos Preview remains the best-aligned model we’ve trained according to our evaluations. Our safety evaluations are discussed in full in the [Claude Opus 4.7 System Card](#).



Overall misaligned behavior score from our automated behavioral audit. On this evaluation, Opus 4.7 is a modest improvement on Opus 4.6 and Sonnet 4.6, but Mythos Preview still shows the lowest rates of misaligned behavior.

Also launching today

In addition to Claude Opus 4.7 itself, we’re launching the following updates:

- *More effort control:* Opus 4.7 introduces a new **xhigh** (“extra high”) [effort level](#) between **high** and **max**, giving users finer control over the tradeoff between reasoning and latency on hard problems. In Claude Code, we’ve raised the default effort level to **xhigh** for all plans. When testing Opus 4.7 for coding and agentic use cases, we recommend starting with **high** or **xhigh** effort.
- *On the Claude Platform (API):* as well as support for higher-resolution images, we’re also launching task budgets in public beta, giving developers a way to guide Claude’s token spend so it can prioritize work across longer runs.

- *In Claude Code*: The new `/ultrareview slash command` produces a dedicated review session that reads through changes and flags bugs and design issues that a careful reviewer would catch. We're giving Pro and Max Claude Code users three free ultrareviews to try it out. In addition, we've extended `auto mode` to Max users. Auto mode is a new permissions option where Claude makes decisions on your behalf, meaning that you can run longer tasks with fewer interruptions—and with less risk than if you had chosen to skip all permissions.

Migrating from Opus 4.6 to Opus 4.7

Opus 4.7 is a direct upgrade to Opus 4.6, but two changes are worth planning for because they affect token usage. First, Opus 4.7 uses an updated tokenizer that improves how the model processes text. The tradeoff is that the same input can map to more tokens—roughly 1.0–1.35× depending on the content type. Second, Opus 4.7 thinks more at higher effort levels, particularly on later turns in agentic settings. This improves its reliability on hard problems, but it does mean it produces more output tokens.

Users can control token usage in various ways: by using the effort parameter, adjusting their task budgets, or prompting the model to be more concise. In our own testing, the net effect is favorable—token usage across all effort levels is improved on an internal coding evaluation, as shown below—but we recommend measuring the difference on real traffic. We've written a [migration guide](#) that provides further advice on upgrading from Opus 4.6 to Opus 4.7.